UNIVERSITÉ Clermont Auvergne

École Universitaire de physique et d'ingénierie

Projet de synthese M2 PAR
Rapport de projet

# Vision neuromorphique pour la navigation d'un robot mobile


***Élèves :***
Nadjib

***Enseignant :***
Encadrant a completer

16 février 2026

# Table des matières

**Introduction** **1**

**1 Matériel et fondements théoriques** **2**
1.1 Présentation du matériel . . . . . 2
1.1.1 Présentation de la caméra DAVIS346 et fiche technique . . . . . 2
1.1.2 Présentation du robot LIMO . . . . . 3
1.2 Concepts théoriques mobilisés . . . . . 3
1.2.1 Caméra à événements : notion, bref historique, avantages et limites 3
1.2.2 Détection d'obstacles (IMO) : panorama rapide des familles . . . . 3
1.2.3 Navigation autonome et algorithme utilisé dans le projet . . . . . 4

**2 Méthodologie** **5**
2.1 Motion compensation . . . . . 5
2.2 Objects segmentation . . . . . 5
2.3 Objects clustering . . . . . 6
2.4 Création d'une layer Nav2 pour les obstacles dynamiques . . . . . 6

**4 Tests et analyse des résultats** **7**
4.1 Protocole expérimental . . . . . 7
4.2 Résultats . . . . . 7
4.3 Analyse . . . . . 7
4.4 Discussion . . . . . 8

**Conclusion** **9**

# Table des figures

# Liste des tableaux

1.1 Fiche technique synthétique de la DAVIS346 . . . . . 2

# Introduction

Ce rapport présente la conception et l'intégration d'une chaîne de perception neuromorphique pour la navigation autonome d'un robot mobile LIMO équipé d'une caméra DAVIS346.

Le cœur du projet est l'implémentation d'une méthode de détection d'objets en mouvement basée sur la compensation d'ego-mouvement par IMU dans un flux d'événements. Cette approche est ensuite raccordée à Nav2 afin d'injecter des obstacles dynamiques dans la prise de décision de navigation.

Le document est organisé comme suit :

— le chapitre 1 introduit le matériel et les fondements théoriques ;
— le chapitre 2 détaille la méthodologie d'implémentation de l'article retenu et son adaptation ROS 2 ;
— le chapitre 4 présente le protocole de test et l'analyse des résultats ;
— la conclusion synthétise les acquis et les perspectives.

# Chapitre 1

# Matériel et fondements théoriques

## 1.1 Présentation du matériel

### 1.1.1 Présentation de la caméra DAVIS346 et fiche technique

La caméra DAVIS346 est un capteur neuromorphique hybride qui produit en parallèle un flux d'événements (DVS) et des images APS classiques sur un même capteur. Cette architecture est intéressante pour notre projet, car elle permet de conserver une information temporelle très fine tout en gardant une référence image exploitable pour le debug et la visualisation [1].

Les caractéristiques techniques directement utiles pour la chaîne de perception du projet sont résumées dans le tableau 1.1.

**Table 1.1** – *Fiche technique synthétique de la DAVIS346*

| Paramètre | Valeur |
|---|---|
| Résolution capteur | 346 x 260 |
| Sorties | Événements DVS + images APS concurrentes |
| Plage dynamique événementielle | Jusqu'à 120 dB |
| Résolution temporelle | 1 us |
| Latence événementielle | Sous 1 us (nominale) |
| Débit maximal | Jusqu'à 12 MEPS |
| IMU intégrée | 6 axes (gyro + accéléromètre), jusqu'à 8 kHz |
| Alimentation | $<$ 180 mA à 5 V |
| Connectique principale | USB 3.0 Micro-B (verrouillable) |
| Monture optique | CS-mount (adaptateur C-mount possible) |

Le document technique signale également des limites pratiques à connaître lors des essais : salves d'événements possibles en mode APS Global Shutter, schéma de balayage DVS sous forte charge, et performances image APS inférieures à des caméras classiques. Ces limites justifient un réglage fin des biais capteur et une calibration expérimentale adaptée au contexte robotique [1].

### 1.1.2 Présentation du robot LIMO

Le robot LIMO d'AgileX est présenté comme une plateforme ROS polyvalente avec quatre modes de locomotion (différentiel, Ackermann, chenilles et mecanum), ce qui en fait un support pertinent pour des tests de navigation dans des scènes hétérogènes [2, 3].

La documentation technique indique en particulier :

— dimensions compactes (322 x 220 x 251 mm) et charge utile de 4 kg ;
— vitesse maximale sans charge de l'ordre de 1 m/s ;
— calcul embarqué basé sur NUC sous Ubuntu 22.04 ;
— capteurs intégrés utiles à la navigation (LiDAR, caméra de profondeur, IMU).

La pile logicielle officielle propose des launches ROS 2 pour le démarrage de la base, la cartographie, puis la navigation Nav2 suivant le mode de conduite retenu [2]. Cette base matérielle et logicielle est compatible avec notre intégration de la perception événementielle.

## 1.2 Concepts théoriques mobilisés

### 1.2.1 Caméra à événements : notion, bref historique, avantages et limites

Une caméra à événements est un capteur asynchrone : chaque pixel déclenche un événement uniquement lors d'une variation de luminance, au lieu d'attendre une image globale échantillonnée à fréquence fixe. La revue de référence rappelle que ce paradigme s'inscrit dans le champ plus large de l'ingénierie neuromorphique et qu'il s'est structuré au cours des dernières décennies avec des familles de capteurs comme DVS ou ATIS [4].

Les avantages majeurs pour la robotique mobile sont :

— forte plage dynamique (ordre de 120 dB) utile en éclairage difficile ;
— très faible latence de perception ;
— réduction de la redondance et de la consommation grâce au codage asynchrone des changements [4].

Les limites relevées dans la même revue sont aussi importantes :

— flux sparse plus difficile à exploiter que des images denses ;
— capteurs encore coûteux ;
— résolution et interfaces encore contraintes sur certains systèmes commerciaux [4].

### 1.2.2 Détection d'obstacles (IMO) : panorama rapide des familles

La détection d'objets en mouvement indépendant (IMO) vise à séparer les mouvements liés à l'ego-mouvement du robot de ceux des objets réels de la scène. Dans un cadre événementiel, la revue distingue trois familles méthodologiques principales [4] :

— approches de compensation de mouvement (soustraction du mouvement global, souvent via flot optique) ;
— approches résiduelles itératives combinant ego-mouvement et segmentation de résidus ;
— approches apprentissage (supervisé ou non supervisé) pour segmenter directement les IMO.

Ce panorama montre que les approches à base de compensation restent attractives pour des contraintes de temps réel embarqué, ce qui motive le choix méthodologique du projet.

### 1.2.3 Navigation autonome et algorithme utilisé dans le projet

La documentation LIMO rappelle le schéma classique de navigation mobile : localisation sur carte + planification globale + planification locale [2]. En pratique, l'exécution de la navigation est faite via Nav2 avec un lancement dédié au mode différentiel (`limo_nav2_diff.launch.py`) [2].

Dans notre configuration, le planificateur global est de type NavFn (équivalent Dijkstra quand l'option A* est inactive), le contrôle local repose sur DWB, et la localisation est assurée par AMCL. Cette architecture sert de base pour injecter ensuite les obstacles dynamiques issus de la perception événementielle.

# Chapitre 2

# Méthodologie

Ce chapitre détaille l'implémentation de l'article *Event-based Real-time Moving Object Detection Based On IMU Ego-motion Compensation* dans notre environnement ROS 2 [5].

## 2.1 Motion compensation

L'étape clé est la compensation de l'ego-mouvement de la caméra. Dans l'article, les auteurs associent des paquets d'événements à des mesures IMU sur une fenêtre temporelle courte, puis appliquent une fonction de warping non linéaire issue d'un modèle de rotation rigide. L'objectif est de réaligner les événements du fond statique pour rendre les zones dynamiques plus visibles [5].

Toujours selon l'article, la formulation non linéaire améliore la précision de compensation (ordre de 10 à 15%) par rapport à une approximation linéaire, tout en restant compatible avec une exécution temps réel embarquée [5].

Sur le plan logiciel, nous sommes partis de la version ROS1 du package de motion compensation puis nous avons réalisé une adaptation ROS 2 dans le package `datasync_3_0`. Le nœud ROS 2 :

- subscribe les événements et l'IMU ;
- calcule une vitesse angulaire moyenne sur fenêtre ;
- produit des événements compensés, une count-image, une time-image et un seuil dynamique $\lambda$ publié sur un topic dédié.

## 2.2 Objects segmentation

Après compensation, le papier exploite le fait que les événements de fond stabilisés se concentrent spatialement, alors que les événements d'objets indépendants restent plus dispersés. La segmentation est faite via une count-image et une time-image : un pixel est classé dynamique si sa valeur temporelle normalisée dépasse un seuil dynamique $\lambda = a\|\omega\| + b$ [5].

Dans notre implémentation ROS 2 (`project_ws/src/event_segmentation`), le nœud de segmentation :

- synchronise `/time_image` et `/count_image` ;
- récupère $\lambda$ depuis `/event_lambda_threshold` ;

— génère un masque binaire des pixels foreground (avec contraintes de minimum d'activité et filtrage morphologique optionnel).

## 2.3 Objects clustering

La dernière étape de l'article regroupe les événements dynamiques en objets. La logique théorique combine un clustering type DBSCAN (robuste au bruit) avec une métrique mêlant position spatio-temporelle 3D et flot optique 2D pour mieux séparer des objets proches dans le plan image [5].

Dans notre chaîne ROS 2 (`project_ws/src/event_clustering`), la version actuelle réalise un regroupement en composantes connexes du masque foreground, complété par un filtrage temporel de persistance des petites composantes. Cette adaptation conserve l'esprit de la méthode (regroupement spatio-temporel robuste en temps réel) tout en restant simple à maintenir dans le pipeline global.

## 2.4 Création d'une layer Nav2 pour les obstacles dynamiques

Les tutoriels officiels Nav2 décrivent deux voies complémentaires pour injecter les obstacles dynamiques dans la navigation :

— créer un plugin de costmap en dérivant `nav2_costmap_2d::Layer` et en implémentant les méthodes `onInitialize()`, `updateBounds()` et `updateCosts()` ;

— exporter ce plugin via `PLUGINLIB_EXPORT_CLASS` et l'activer dans la liste `plugins` de la costmap [6].

La documentation de configuration Nav2 rappelle aussi que les couches de costmap peuvent ingérer des sources capteurs via `observation_sources`, notamment en `LaserScan` ou `PointCloud2` [7].

Dans notre projet, l'intégration opérationnelle est faite par cette seconde voie : publication des obstacles dynamiques au format `PointCloud2` sur `/dynamic_obstacles`, puis déclaration de cette source dans les costmaps locale et globale (fichier `nav2_diff.yaml`).

# Chapitre 4

# Tests et analyse des résultats

## 4.1 Protocole expérimental

L'évaluation est structurée en quatre blocs, alignés sur la chaîne de traitement complète :

— tests de compensation de mouvement (stabilité du fond après warping) ;
— tests de segmentation (qualité du masque foreground) ;
— tests de clustering (cohérence des objets détectés) ;
— tests d'intégration navigation (prise en compte des obstacles dynamiques dans Nav2).

Les scénarios incluent des phases statiques, des rotations du robot, puis des déplacements avec un ou plusieurs objets en mouvement. Les indicateurs suivis restent volontairement simples :

— densité visuelle des événements compensés sur le fond ;
— taux de bruit résiduel dans le masque ;
— stabilité des contours d'objets sur des séquences consécutives ;
— impact sur la trajectoire Nav2 (détours, arrêts de sécurité, absence de collision).

## 4.2 Résultats

Les essais fonctionnels confirment le comportement attendu de la pipeline :

— la compensation IMU réduit la dispersion des événements de fond et rend les zones dynamiques plus discriminantes ;
— la segmentation par seuil dynamique produit un masque exploitable même quand la scène contient des changements rapides ;
— le clustering permet de convertir le masque en objets utilisables pour la navigation.

Sur le volet navigation, l'ajout de la source `/dynamic_obstacles` dans les costmaps locale et globale permet de refléter les obstacles détectés dans la carte de coût et d'influencer le plan local en temps réel.

## 4.3 Analyse

L'analyse met en évidence trois points :

— le couplage IMU + événement est essentiel : sans compensation, la segmentation dégrade rapidement dès que l'ego-mouvement augmente ;
— le seuil dynamique $\lambda$ est un paramètre sensible qui conditionne le compromis entre bruit et rappel d'objets ;
— l'intégration Nav2 via les costmaps est robuste tant que le topic `/dynamic_obstacles` est publié de façon stable.

Les limites actuelles concernent surtout les cas de faible texture, les objets très proches les uns des autres, et les scènes à fort bruit événementiel.

## 4.4 Discussion

La solution retenue est cohérente avec l'objectif du projet : obtenir une perception d'obstacles dynamiques compatible avec des contraintes temps réel sur robot mobile.

Les pistes d'amélioration prioritaires sont :
— calibration plus fine des paramètres de compensation et de segmentation selon les conditions lumineuses ;
— enrichissement du clustering (métriques mouvement plus explicites) ;
— étape supplémentaire d'association multi-frames pour stabiliser les objets et estimer leur vitesse avant projection dans Nav2.

# Conclusion

Ce rapport a présenté une chaîne complète allant de la perception événementielle à l'intégration navigation sur robot mobile. Le matériel (DAVIS346 + LIMO), les fondements théoriques, puis les choix d'implémentation ROS 2 ont été explicités de manière cohérente avec l'article de référence.

La contribution principale est l'adaptation d'une approche de détection d'objets dynamiques basée sur compensation IMU, segmentation temporelle et clustering, puis son raccordement à Nav2 via l'injection d'obstacles dynamiques dans les costmaps.

Les suites naturelles du travail sont :

— renforcer la robustesse en scènes fortement dynamiques et peu texturées ;
— raffiner le suivi temporel des objets détectés ;
— consolider l'évaluation quantitative (latence, précision détection, impact navigation) sur des campagnes de tests plus longues.

# Bibliographie


[1] iniVation AG. *DAVIS346 Datasheet*, 2025. Release 2025-08-05, technical documentation.

[2] AgileX Robotics. Limo ros2 humble documentation. `https://raw.githubusercontent.com/agilexrobotics/limo_ros2_doc/master/LIMO-ROS2-humble%28EN%29.md`, 2026. Accessed on 2026-02-16.

[3] AgileX Robotics. Limo, the first multi-modal mobile robot with ai modules and ros demo. `https://www.youtube.com/watch?v=G19B7R4gIGU`, 2026. Video presentation, accessed on 2026-02-16.

[4] A. Novoa, F. Lobon, H. G. De Marina, S. Romero, and F. Barranco. Neuromorphic perception and navigation for mobile robots : A review. *arXiv preprint arXiv :2407.06792*, 2024.

[5] Chunhui Zhao, Yakun Li, and Yang Lyu. Event-based real-time moving object detection based on imu ego-motion compensation. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 690–696, 2023.

[6] Nav2 Documentation. Writing a new costmap2d plugin. `https://docs.nav2.org/plugin_tutorials/docs/writing_new_costmap2d_plugin.html`, 2026. Accessed on 2026-02-16.

[7] Nav2 Documentation. Configuring costmaps. `https://docs.nav2.org/configuration/packages/configuring-costmaps.html`, 2026. Accessed on 2026-02-16.